

TSOpt guide 1: Your data in

Contents

1	The example data	2
2	Reading any format: <code>tso_read()</code>	2
2.1	Spreadsheets and other tables	4
2.2	Polyploids	4
3	Quality control: <code>tso_qc()</code>	4
3.1	Base frequencies and why they matter across cycles	5
3.2	A note on allele orientation	6
4	Relationship matrices: <code>tso_kinship()</code>	6
5	Beyond additive: <code>tso_kernels()</code>	7
6	Writing and converting: <code>tso_write()</code>	8
7	One object for everything: <code>tso_data()</code>	8
7.1	Groups, families and costs	9
7.2	Breeding systems	10
7.3	No genotypes? A kinship matrix or a pedigree	11
7.4	Environments, traits and major genes	11
8	Straight from your breeding database (BrAPI)	11
9	Common problems	12
10	Next	13

This is the first of five guides that take you from a genotype file to a finished training-set plan. This one covers everything that happens before any design: reading the file your genotyping provider sent, cleaning it, building the relationship matrix and packing it all into the one object the rest of TSOpt works on.

Before anything else: your licence key. TSOpt is free of charge for everyone, but each user needs a personal licence key from PlantNura. `tso_licence_request()` writes the request email; activate the key you receive once with `tso_licence("TS01...")`. It is checked offline, saved on your computer and found automatically from then on; `tso_licence()` shows yours. Nothing is ever sent from your computer.

The whole chain is four functions, and `tso_data()` runs all of them for you:

Step	Function	What it does
1	<code>tso_read()</code>	any genotype file -> a lines x markers dosage matrix
2	<code>tso_qc()</code>	drops poor markers and lines, imputes gaps, stores base frequencies
3	<code>tso_kinship()</code>	the relationship matrix K that every design is computed on

Step	Function	What it does
4	<code>tso_data()</code>	steps 1-3, plus breeding system, groups, families and costs

For most users the whole guide collapses into one line, `tp <- tso_data("my_panel.vcf.gz")`. The sections below show what that line does, so that you can check it and change it when your data need something else. If your genotypes and trials live in a breeding database (BMS, Breedbase, GIGWA), the last section before *Common problems* reads them straight from it through BrAPI, with no file in between.

1 The example data

TSOpt ships one simulated panel of 300 inbred lines and 1,000 SNPs, written in five formats, plus a few side tables. Because it is simulated, the true genetic values are known, which later guides use to check how good a design really was.

```
library(TSOpt)
ex <- function(f) system.file("extdata", f, package = "TSOpt")
list.files(system.file("extdata", package = "TSOpt"))
#> [1] "example_costs.csv"          "example_dart.csv"
#> [3] "example_groups.csv"        "example_marker_effects.csv"
#> [5] "example_panel.csv.gz"      "example_panel.hmp.txt.gz"
#> [7] "example_panel.vcf.gz"      "example_pheno.csv"
#> [9] "example_plink.bed"         "example_plink.bim"
#> [11] "example_plink.fam"         "example_true_values.csv"
```

File	Contents
example_panel.vcf.gz, .hmp.txt.gz, .csv.gz	the panel as VCF, HapMap and a CSV of 0/1/2
example_plink.bed/.bim/.fam	the same panel as PLINK binary
example_dart.csv	a smaller DArTseq-style file (60 lines x 200 markers)
example_pheno.csv	id, y: one phenotype per line
example_true_values.csv	id, g: the true genetic values
example_groups.csv	id, group: four subpopulations
example_costs.csv	id, cost: the cost of phenotyping each line

2 Reading any format: `tso_read()`

`tso_read()` looks at the content of the file, not only its name, so a VCF saved as `.txt` is still read as a VCF. Compressed files (`.gz`, `.bz2`, `.xz`, `.zip`) need no unpacking.

```
g_vcf <- tso_read(ex("example_panel.vcf.gz"))
#> Read example_panel.vcf.gz (VCF) -> 300 lines x 1,000 markers
#> Coding VCF GT -> ALT-allele count 0..2 · ploidy 2 (detected)
#> Missing 0.0%
```

Every reader prints three lines, and they are worth reading each time:

- **Read:** the file, the format it was recognised as, and the size. Check that the number of lines and markers is what you expect. If they are swapped, see *Common problems* below.

- **Coding:** how the genotypes were turned into numbers, and which allele is counted. Here a VCF GT call becomes the count of the ALT allele, 0 to 2, and the ploidy was detected from the calls.
- **Missing:** the share of missing calls. The example has none; a real GBS file often has 10-30%.

The same panel in the other formats:

```
g_hmp <- tso_read(ex("example_panel.hmp.txt.gz"), verbose = FALSE)
g_csv <- tso_read(ex("example_panel.csv.gz"), verbose = FALSE)
g_bed <- tso_read(sub("\\.bed$", "", ex("example_plink.bed")), verbose = FALSE)
g_dart <- tso_read(ex("example_dart.csv"), verbose = FALSE)
all_g <- list(VCF = g_vcf, HapMap = g_hmp, CSV = g_csv, PLINK = g_bed, DArT = g_dart)
data.frame(read_as = sapply(all_g, `[`, "format"),
            lines = sapply(all_g, function(x) nrow(x$dosage)),
            markers = sapply(all_g, function(x) ncol(x$dosage)))
#>               read_as lines markers
#> VCF                  VCF    300   1000
#> HapMap              HapMap    300   1000
#> CSV delimited table    300   1000
#> PLINK      PLINK binary    300   1000
#> DArT      DArTseq one-row    60    200
```

For PLINK binary, give the prefix (the path without .bed) or any one of the three files. The full list of what is recognised:

You have	Write
VCF, compressed or not	<code>tso_read("panel.vcf.gz")</code>
TASSEL / GBS HapMap	<code>tso_read("panel.hmp.txt")</code>
PLINK binary	<code>tso_read("cohort")</code> (prefix of .bed/.bim/.fam)
PLINK text / --recode A	<code>tso_read("cohort.ped") /</code> <code>tso_read("cohort.raw")</code>
DArTseq, one-row or two-row	<code>tso_read("Report_DArT.csv")</code>
CSV, TSV or TXT table	<code>tso_read("genotypes.csv")</code>
Excel	<code>tso_read("genotypes.xlsx")</code> (needs the readxl package)
R file or R object	<code>tso_read("geno.rds"),</code> <code>tso_read(my_matrix)</code>

The result is a `tso_geno` object: a list with the dosage matrix (lines in rows, markers in columns), a marker map, the ploidy, the format and the log.

```
g_vcf$dosage[1:3, 1:6]
#>      m1 m2 m3 m4 m5 m6
#> L001  2  2  0  2  2  0
#> L002  0  0  0  0  0  2
#> L003  2  2  0  0  2  0
head(g_vcf$map, 3)
#>   marker chrom   pos counted other
#> 1     m1     1 1e+05         A     C
#> 2     m2     1 2e+05         A     C
#> 3     m3     1 3e+05         A     C
```

2.1 Spreadsheets and other tables

Tables can use any coding: 0/1/2, -1/0/1, 0/1 for inbred lines, 0 to ploidy for polyploids, decimal dosages, AA/AB/BB, nucleotide pairs (AG, A/G, A|G) or single IUPAC letters. Missing codes such as NA, N, -, --, ., ./., -9 and the Axiom -1 are all recognised.

```
m <- matrix(c("AA", "AG", "GG", "A/A", "A/G", ". / .", "A", "R", "G"), 3,
            dimnames = list(paste0("L", 1:3), c("snp1", "snp2", "snp3")))
tso_read(m, markers_in = "columns")$dosage
#> Read R object (R object) -> 3 lines x 3 markers
#> Coding allele calls -> minor-allele count 0/1/2
#> Missing 11.1%
#>      snp1 snp2 snp3
#> L1      0      0      0
#> L2      1      1      1
#> L3      2     NA      2
```

The options that matter for tables and VCFs:

Argument	Use it when
markers_in = "columns" or "rows"	the table's layout is not detected correctly (the default "auto" assumes more markers than lines)
id_col = "Genotype"	the line IDs are not in the first column
binary_as = "dosage"	a 0/1 table really means 0/1 copies, not 0/2 homozygotes (the default for inbred lines)
vcf_field = "DS"	you want imputed dosages from the VCF instead of the hard GT calls
ploidy = 4	the ploidy cannot be read from the data (a tetraploid with no triplex calls, say)
format = "vcf"	you want to overrule the automatic detection

2.2 Polyploids

A table of 0 to 4 is read as tetraploid without being told. Everything downstream, from QC to the relationship matrix, then uses the ploidy.

```
set.seed(1)
tet <- matrix(sample(0:4, 40 * 100, replace = TRUE), 40,
              dimnames = list(sprintf("C%02d", 1:40), paste0("snp", 1:100)))
g_tet <- tso_read(tet, markers_in = "columns")
#> Read R object (R object) -> 40 lines x 100 markers
#> Coding 0..4 dosage · ploidy 4
#> Missing 0.0%
```

If your polyploid file happens to contain only 0, 1 and 2 at the markers you sent, the ploidy would be guessed as 2: always give ploidy = for polyploid crops.

3 Quality control: tso_qc()

tso_qc() removes markers and lines that would add noise to the relationship matrix, fills the remaining gaps, and stores the allele frequencies of this panel. Its arguments are all you need to know:

Argument	Default	Meaning
maf	0.05	drop markers whose minor-allele frequency is below this
call_rate	0.8	drop markers called in fewer than 80% of lines
line_call_rate	0.5	drop lines called at fewer than 50% of markers
impute	"mean"	replace remaining missing calls by the marker mean ("none" keeps them)
base_freq	NULL	allele frequencies of an earlier base population to keep fixed
verbose	TRUE	print the log

The example panel has no missing calls, which is unrealistic. To see every step at work we blank out some calls first: 30 markers with 30% missing, 200 markers with 5% missing, and two lines that failed.

```
g_raw <- g_vcf
set.seed(2)
cols <- sample(ncol(g_raw$dosage), 230)
for (j in cols[1:30]) g_raw$dosage[sample(300, 90), j] <- NA
for (j in cols[31:230]) g_raw$dosage[sample(300, 15), j] <- NA
g_raw$dosage[c("L010", "L020"), sample(1000, 700)] <- NA
g <- tso_qc(g_raw)
#> QC dropped 2 lines with call rate < 0.5
#> QC dropped 72 markers (MAF < 0.05: 42; call rate < 0.8: 30)
#> Hets 0.0% heterozygous calls
#> Impute 1.1% missing calls replaced by the marker mean
#> Base freq estimated from these 298 lines and stored (reused for later cycles)
```

Line by line:

- **QC dropped 2 lines:** L010 and L020 were called at under half the markers. A line with that few calls has an unreliable place in the relationship matrix.
- **QC dropped ... markers:** rare markers (MAF < 0.05) carry little information about relationships and are the most error-prone; markers missing in more than 20% of lines are dropped for the same reason. The two counts in brackets tell you which rule did the work.
- **Hets:** the share of heterozygous calls. For doubled haploids or inbred lines this should be close to 0; several percent means residual heterozygosity, seed mix-ups or calling errors. Clones and outcrossers are expected to be heterozygous.
- **Impute:** the share of calls still missing after filtering, now replaced by the marker mean. Mean imputation is adequate for a relationship matrix; impute with Beagle or a similar tool first if you also want the calls for GWAS.
- **Base freq:** the allele frequencies are stored in the object.

Every step is also appended to `g$log` (print it with `cat(g$log, sep = "\n")`), so the object always carries its own history, from the file it was read from onwards.

What to look at: 298 lines and 928 markers are left. Always check the line count: a line dropped here cannot be designed for later.

3.1 Base frequencies and why they matter across cycles

The relationship matrix centres each marker on its allele frequency. If next year's matrix is centred on next year's frequencies, the same two lines get a different relationship and last year's plan is no longer comparable. `tso_qc()` therefore stores the frequencies (`g$base_freq`), and `tso_kinship()` and `tso_update()` reuse them. Next season, pass them on:

```
g_2027 <- tso_qc("panel_2027.vcf.gz", base_freq = g$base_freq)
```

With `base_freq` given, a marker is kept when it was polymorphic in the base population, and the log says “Base freq supplied by the user (kept fixed)”.

3.2 A note on allele orientation

Formats disagree on which allele they count. VCF counts the ALT allele, PLINK the A1 allele, and HapMap and letter tables count each file’s own minor allele. The same marker can therefore be coded 0 in one file and 2 in another. The stored base frequencies remember which allele was counted, so the next file is turned the right way round before anything else. Here the HapMap copy of the panel is aligned to the VCF:

```
g_ref <- tso_qc(g_vcf, verbose = FALSE)
g_h <- tso_qc(g_hmp, base_freq = g_ref$base_freq, verbose = FALSE)
grep("Alleles", g_h$log, value = TRUE)
#> [1] "Alleles 471 markers re-oriented to the base population's counted allele"
max(abs(tso_kinship(g_ref) - tso_kinship(g_h)))
#> [1] 0
```

471 markers were flipped, and the two relationship matrices are then identical. Without `base_freq` each file is simply read on its own terms, which is fine within one season.

4 Relationship matrices: `tso_kinship()`

The design engine works on a relationship matrix `K` between lines. `tso_kinship()` builds it three ways:

- `method = "additive"` (default): VanRaden’s genomic relationship matrix, for any ploidy.
- `method = "dominance"`: the dominance relationship matrix (Vitezica et al. 2013), for diploids. Useful for clones and hybrids.
- `method = "pedigree"`: the numerator relationship matrix `A` from a pedigree table.

```
K <- tso_kinship(g)
round(K[1:4, 1:4], 3)
#>      L001 L002 L003 L004
#> L001 1.000 0.044 0.114 0.085
#> L002 0.044 0.983 0.080 0.307
#> L003 0.114 0.080 0.989 0.074
#> L004 0.085 0.307 0.074 0.946
```

By default `K` is scaled to a mean diagonal of 1 (`scale = TRUE`). This is why you can give TSOpt the entry-mean heritability of your trial without worrying about the ploidy or the inbreeding of the panel: doubled haploids have a raw diagonal near 2, which the scaling absorbs.

```
c(raw = mean(diag(tso_kinship(g, scale = FALSE))), scaled = mean(diag(K)))
#>      raw      scaled
#> 1.979046 1.000100
```

A small ridge ($1e-4$) is added to the diagonal for numerical stability. The dominance kernel and the tetraploid case work the same way:

```
Kd <- tso_kinship(g, method = "dominance")
K4 <- tso_kinship(tso_qc(g_tet, verbose = FALSE))
dim(K4)
#> [1] 40 40
```

A pedigree is a data frame of id, parent 1 and parent 2, with unknown parents as NA or 0. Here two full sibs X1 and X2, and Y1 from X1 crossed with an unrelated F3:

```
ped <- data.frame(id = c("F1", "F2", "F3", "X1", "X2", "Y1"),
                  p1 = c(NA, NA, NA, "F1", "F1", "X1"),
                  p2 = c(NA, NA, NA, "F2", "F2", "F3"))
tso_kinship(method = "pedigree", pedigree = ped, scale = FALSE, ridge = 0)
#>      F1  F2  F3  X1  X2  Y1
#> F1 1.00 0.00 0.0 0.5 0.50 0.25
#> F2 0.00 1.00 0.0 0.5 0.50 0.25
#> F3 0.00 0.00 1.0 0.0 0.00 0.50
#> X1 0.50 0.50 0.0 1.0 0.50 0.50
#> X2 0.50 0.50 0.0 0.5 1.00 0.25
#> Y1 0.25 0.25 0.5 0.5 0.25 1.00
```

Full sibs share 0.5 and Y1 shares 0.5 with each of its parents, as expected.

Weighted markers. With `weights` = some markers count more than others, for example markers near known QTL (weights such as `-log10 p` from `tso_major_scan()`), or weight 0 to leave out loci that you model separately. Markers without a weight get 1.

```
w <- setNames(rep(5, 10), colnames(g$dosage)[1:10]) # first 10 markers count 5 times
Kw <- tso_kinship(g, weights = w)
cor(c(K), c(Kw))
#> [1] 0.9941339
```

Guide 4 shows when this, or the `major` = option of `tso_data()`, is the better choice.

5 Beyond additive: `tso_kernels()`

The additive matrix assumes all genetic variance is additive. `tso_kernels()` builds the other covariance models a breeder may need, each scaled to mean diagonal 1:

- **From markers.** A (additive), D (dominance, diploids), the epistatic products AA, AD and DD (element-wise products of A and D), and the non-parametric gaussian and exponential kernels (RKHS), which capture additive and epistatic similarity together.
- **From a pedigree.** A_ped, D_ped (the dominance relationship: full sibs share it, half sibs and parent and offspring do not) and AA_ped.
- **Both.** H, the single-step matrix, for a pedigree in which only some individuals are genotyped. Ungenotyped individuals, for example next season's crosses, get their relationships through the pedigree, conditioned on the genotyped ones.

```
g_out <- tso_qc(tso_simulate_panel(n = 80, markers = 500, type = "outbred", seed = 3),
               verbose = FALSE)
kk <- tso_kernels(g_out, which = c("A", "D", "AA", "gaussian"))
kk
#> TSOpt kernels • 4 • each scaled to mean diagonal 1
#> A additive 80 lines additive genomic relationship (VanRaden
#> 2008)
#> D dominance 80 lines dominance genomic relationship (Vitezica
#> et al. 2013; diploids)
#> AA epistatic 80 lines additive x additive epistasis, A # A
#> (Henderson 1985; Jiang & Reif 2015)
#> gaussian non-parametric 80 lines Gaussian kernel exp(-d^2 / h), RKHS
#> (Gianola & van Kaam 2008)
#> Next: tso_data(geno, kernels = kernels, kernel_share = "robust")
```

Epistatic relationships fall off faster than additive ones, because they are roughly the square

of them: distant relatives share almost no epistatic value.

```
off <- upper.tri(kk$K$A)
round(c(A = mean(abs(kk$K$A[off])), AA = mean(abs(kk$K$AA[off]))), 3)
#>      A      AA
#> 0.158 0.036
```

Which kernels make sense depends on the material. Inbred lines and doubled haploids carry no heterozygotes, so there is no dominance to model: use A and AA (TSOpt warns if you ask for D on inbred material). Clones inherit everything, so A, D and AA are all relevant. Hybrids already model dominance and epistasis between the pools through SCA.

The kernels go to `tso_data()`, which combines them with the share of genetic variance each explains. Shares are rarely known well (epistatic variance is notoriously hard to estimate), so the default is `kernel_share = "robust"`: the design averages over non-additive shares of 10%, 30% and 50%, the way it already averages over heritability.

```
tp_k <- tso_data(g_out, kernels = c("A", "AA"), verbose = FALSE)
tp_k$kernels$share # the centre of the robust scenarios
#>      A      AA
#> 0.7 0.3
tp_f <- tso_data(g_out, kernels = kk, kernel_share = c(A = 0.6, D = 0.2, AA = 0.1,
  ↪ gaussian = 0.1),
  verbose = FALSE)
```

Your own matrices work too: `kernels = list(A = my_grm, E = my_kernel)`. Name the additive one "A" so that TSOpt can design for breeding values (guide 2).

6 Writing and converting: `tso_write()`

`tso_write()` saves a `tso_geno` object (or a plain dosage matrix) in the format given by the file extension. Add `.gz` to compress text formats. That makes it a converter too: read a HapMap, write PLINK.

```
dir <- tempdir()
for (ext in c(".vcf.gz", ".hmp.txt", ".bed", ".raw", ".csv")) {
  f <- file.path(dir, paste0("panel", ext))
  tso_write(g, f)
  back <- tso_read(if (ext == ".bed") sub("\\.bed$", "", f) else f, verbose = FALSE)
  cat(sprintf("%-9s %d lines x %d markers (%s)\n", ext,
    nrow(back$dosage), ncol(back$dosage), back$format))
}
#> .vcf.gz    298 lines x 928 markers (VCF)
#> .hmp.txt   298 lines x 928 markers (HapMap)
#> .bed       298 lines x 928 markers (PLINK binary)
#> .raw       298 lines x 928 markers (PLINK additive (.raw))
#> .csv       298 lines x 928 markers (delimited table)
```

HapMap and PLINK binary are written for diploids only; VCF, CSV and `.rds` also take polyploids. `.rds` keeps the whole object, including the log and the base frequencies.

7 One object for everything: `tso_data()`

`tso_data()` is the front door of TSOpt. It reads the file, runs `tso_qc()` with its defaults, builds the relationship matrix, and sets up the **units**: the things you can phenotype. For inbred lines a unit is a line.

```
tp <- tso_data(ex("example_panel.vcf.gz"))
tp
#> <tso_data> system: inbred · 300 units
#> Read example_panel.vcf.gz (VCF) -> 300 lines x 1,000 markers
#> Coding VCF GT -> ALT-allele count 0..2 · ploidy 2 (detected)
#> Missing 0.0%
#> QC dropped 42 markers (MAF < 0.05: 42; call rate < 0.8: 0)
#> Hets 0.0% heterozygous calls
#> Base freq estimated from these 300 lines and stored (reused for later cycles)
#> Kinship additive (VanRaden, base frequencies), mean diagonal scaled to 1
```

Printing the object shows its log: the read and QC steps you saw above, followed by the kinship line. Here QC dropped only the 42 rare markers, because the shipped file has no missing calls. To change the QC, pass its arguments as a list; to skip it (because you already cleaned the data), use `qc = FALSE`:

```
tp <- tso_data("panel.vcf.gz", qc = list(maf = 0.02, call_rate = 0.9))
tp <- tso_data(g, qc = FALSE) # g was cleaned with tso_qc() already
```

The units table holds one row per unit, and it is where groups, families and costs go:

```
head(tp$units, 3)
#>   id line group family env line_id cost crossable
#> 1 L001 L001 <NA> <NA> <NA> L001 1 TRUE
#> 2 L002 L002 <NA> <NA> <NA> L002 1 TRUE
#> 3 L003 L003 <NA> <NA> <NA> L003 1 TRUE
```

7.1 Groups, families and costs

Groups (subpopulations, programmes, heterotic groups) and families are given as vectors named by line ID; costs are named by unit ID. A line whose ID is not found in the groups vector silently gets no group (NA), while a missing cost is an error, so check that the names match (see *Common problems*).

```
grp <- read.csv(ex("example_groups.csv"))
costs <- read.csv(ex("example_costs.csv"))
tp <- tso_data(ex("example_panel.vcf.gz"),
              groups = setNames(grp$group, grp$id),
              cost = setNames(costs$cost, costs$id), verbose = FALSE)
tp
#> <tso_data> system: inbred · 300 units · 4 groups
#> Read example_panel.vcf.gz (VCF) -> 300 lines x 1,000 markers
#> Coding VCF GT -> ALT-allele count 0..2 · ploidy 2 (detected)
#> Missing 0.0%
#> QC dropped 42 markers (MAF < 0.05: 42; call rate < 0.8: 0)
#> Hets 0.0% heterozygous calls
#> Base freq estimated from these 300 lines and stored (reused for later cycles)
#> Kinship additive (VanRaden, base frequencies), mean diagonal scaled to 1
table(tp$units$group)
#>
#> G1 G2 G3 G4
#> 75 75 75 75
```

Groups are used for stratification, quotas (`group_quota`) and figures. Families feed `max_per_family` in `tso_design()`. Costs let you design to a budget instead of a count: guide 3 shows both.

A quick look at the panel before any design:

`tso_plot(tp)`

What to look at: four clear clusters. The panel is structured, and `tso_design()` will detect this and stratify by itself (guide 2).

7.2 Breeding systems

`system` = tells TSOpt what a unit is and how its genetic value is built:

system	For	A unit is	What changes
"inbred" (default)	lines, doubled haploids, RILs	a line	additive K
"clonal"	potato, cassava, sugarcane, banana	a clone	additive + dominance for diploids (<code>dom_share = 0.2</code>)
"outcross"	families or individuals of outcrossers	an individual	additive K
"genebank"	accessions of a diversity panel	an accession	default design is an untargeted, stratified reference panel
"hybrid"	single crosses between two pools	a hybrid female x male	GCA + GCA + SCA built from the parents

```
ids <- rownames(g_vcf$dosage)
tp_clone <- tso_data(g_vcf, system = "clonal", verbose = FALSE)
tail(tp_clone$log, 1)
#> [1] "Kinship additive + dominance (dominance share 0.2)"
```

For hybrids, give the two parent pools. Every possible cross becomes a unit, and its relationships come from the parents' relationships ($sca_share = 0.2$ of the hybrid variance is SCA, the rest GCA split equally). `crossable` = limits which parents may be used in training crosses; hybrids of other parents can still be predicted.

```
tp_hyb <- tso_data(g_vcf, system = "hybrid",
                  pools = list(ids[1:40], ids[151:190]), verbose = FALSE)
tail(tp_hyb$log, 1)
#> [1] "Hybrids 40 x 40 parents in 2 groups -> 1,600 possible hybrids (SCA share 0.2)"
head(tp_hyb$units[, c("id", "parent1", "parent2")], 3)
#>      id parent1 parent2
#> 1 L001 x L151    L001    L151
#> 2 L001 x L152    L001    L152
#> 3 L001 x L153    L001    L153
```

7.3 No genotypes? A kinship matrix or a pedigree

You can start from a relationship matrix you built elsewhere (`kinship` =, it is rescaled to mean diagonal 1 and checked for being positive semi-definite), or from a pedigree alone:

```
tso_data(pedigree = ped)
#> <tso_data> system: inbred · 6 units
#> Kinship pedigree A (6 individuals)
```

7.4 Environments, traits and major genes

`tso_data()` also takes `envs` and `env_cor` (sparse multi-environment testing), `traits`, `trait_h2` and `gcor` (multi-trait designs) and `major` (protecting major-gene alleles). Each of them changes what a unit is. With `envs`, a unit is a line in one environment; with `traits`, one trait measured on one line. Give both and a unit is one trait on one line in one environment:

```
tp_te <- tso_data(g_vcf, traits = c("yield", "ndvi"), trait_h2 = c(0.3, 0.6),
                 trait_cost = c(yield = 20, ndvi = 0.5), plot_cost = 10,
                 envs = c("E1", "E2"), env_cor = 0.6, verbose = FALSE)
nrow(tp_te$units)
#> [1] 1200
head(tp_te$units[, c("id", "line_id", "trait", "env", "cost")], 3)
#>      id line_id trait env cost
#> 1 L001:yield@E1    L001 yield  E1   20
#> 2 L002:yield@E1    L002 yield  E1   20
#> 3 L003:yield@E1    L003 yield  E1   20
```

The plot cost (`plot_cost`) is paid once for each line grown in each environment, however many traits are then measured on that plot. These options are covered in **TSOpt guide 3: Designing for your programme** and **TSOpt guide 4: Major genes, haplotypes and fixed effects**.

8 Straight from your breeding database (BrAPI)

Many programmes keep their genotypes and trial data in a breeding database: BMS, Breedbase, GIGWA, DeltaBreed and others. Most of them speak BrAPI v2, the Breeding API, and TSOpt can read from any BrAPI v2 server directly. There is no export, no file and no reformatting. The connector needs the `curl` and `jsonlite` packages (`install.packages(c("curl", "jsonlite"))`).

The code below needs a network connection and a server, so it is shown but not run here. The address is the public BrAPI test server; use your own database's address and, if it needs a login, the access token it gives you.

```
# 1. Connect. /brapi/v2 is added to the address if it is missing.
con <- tso_brapi_connect("https://test-server.brapi.org",
                        token = Sys.getenv("BRAPI_TOKEN")) # "" sends no token

# 2. See what is there: trials (studies) and genotyping data sets (variant sets).
studies <- tso_brapi_studies(con, commonCropName = "Wheat")
vsets <- tso_brapi_variantsets(con)
vsets[, c("variantSetDbId", "variantSetName", "callSetCount", "variantCount")]

# 3. Genotypes of one variant set, as a tso_geno object (like tso_read()).
geno <- tso_brapi_geno(con, variant_set = vsets$variantSetDbId[1])

# 4. One trait as germplasm means, named by germplasm like the genotypes.
y <- tso_brapi_trait(con, study = studies$studyDbId[1], trait = "Grain yield")

# 5. From here on it is the ordinary workflow.
tp <- tso_data(geno)
plan <- tso_design(tp, n = 60, target = setdiff(tp$units$id, names(y)),
                  history = intersect(names(y), tp$units$id))
```

What each step does:

1. **tso_brapi_connect()** reads the server's /serverinfo to learn which services it offers, and keeps the token, page size, time-out and retry settings. Paging and asynchronous searches are handled for you.
2. **tso_brapi_studies()** and **tso_brapi_variantsets()** list the trials and the genotyping data sets, with their IDs. Filters such as commonCropName, programDbId or studyDbId are passed on to the server.
3. **tso_brapi_geno()** downloads the calls (through /allelematrix when the server has it, otherwise /variantsets/{id}/calls), turns strings such as 0/1, 1|1 or 0/0/1/1 into dosages of any ploidy, and names the rows by germplasm. The result goes into **tso_qc()**, **tso_kinship()** or **tso_data()** like any file.
4. **tso_brapi_trait()** averages the observations of one trait per germplasm (a plain mean over plots and studies). For the whole long table, one row per observation, use **tso_brapi_pheno()**; compute adjusted means from it when study effects are strong.
5. The phenotyped lines are last season's **history** and the unphenotyped ones the **targets** (guide 3); y itself is what **tso_evaluate()** and **tso_next_batch()** use (guide 5).

Genotypes and phenotypes are matched by germplasm name. If your database has several germplasm records with the same name, use `names = "germplasmDbId"` in **tso_brapi_geno()** and `id = "germplasmDbId"` in **tso_brapi_pheno()** on both sides.

9 Common problems

Lines came out as columns. If the Read line reports, say, 1,000 lines x 300 markers, the table layout was guessed wrong. The automatic rule assumes there are more markers than lines; a small marker panel on many lines breaks it. Give the layout:

```
small <- t(g_vcf$dosage[1:50, 1:20]) # 20 markers (rows) x 50 lines (columns)
dim(tso_read(small, verbose = FALSE)$dosage) # guessed wrong
#> [1] 20 50
dim(tso_read(small, markers_in = "rows", verbose = FALSE)$dosage) # right
#> [1] 50 20
```

IDs do not match. Group, family, cost and phenotype tables are matched by line ID, so "L001", "l001", "L001 " and "L1" are all different. Compare them before building the object:

```
setdiff(grp$id, rownames(g_vcf$dosage))    # should be empty
#> character(0)
```

Common causes are trailing spaces, VCF sample names with a plate suffix (L001:A01), and Excel turning IDs into numbers or dates. Fix the names with `trimws()` or `sub()` and check again.

Missing codes are read as genotypes. The usual codes are recognised, but a provider may use its own ("?", "0/0" meaning missing, 9). If the Missing line says 0% where you know there are gaps, or the Coding line shows an unexpected range, replace the code with NA in R before calling `tso_read()`.

Polyploids read as diploid. The ploidy is guessed from the largest dosage present. Give `ploidy = 4` (or 6, 8) to `tso_read()` or `tso_data()`. The dominance kernel is for diploids only; for polyploid clones TSOpt uses the additive kernel and says so in the log.

All markers dropped. `tso_qc()` stops if fewer than two markers pass. This usually means the coding was misread (every marker looks monomorphic) or a line-in-columns table; check the Coding and Read lines first, then relax `maf` or `call_rate`.

10 Next

You now have `tp`, the object every design starts from. **TSOpt guide 2: Your first training set** uses it to decide whether optimisation is worth it, how many lines to phenotype, and which ones. The algebra behind the relationship matrix and `CDmean` is worked through in base R in the vignette *CDmean by hand, then with TSOpt* (`vignette("by-hand", package = "TSOpt")`).